- At each step, we use the distance of the `m`-th closest point to draw our NN perimeter, and prune search;
- To keep track of these `m` distances, we can use a bounded priority queue. We have described something similar in chapter 2, section 2.7.3, when we described a method to find the `m` largest values in a stream of numbers.

Listing 9.10 details the pseudo-code for the `nNearestNeighbor` method.

Listing 9.10 The nNearestNeighbor method

```
function nNearestNeighbor(node, target,
n)                              #1
  pq ← new BoundedPriorityQueue(n)
#2
  pq.insert((inf,
null))                                                  #3
  pq ← nNearestNeighbor(node, target,
pq)                              #4
  (nnnDist, _) ← pq.peek()
#5
  if nnnDist == inf then
#6
    pq.top()
  return pq
                  #7

function nNearestNeighbor(node, target, pq)
                  #8
  if node == null
then                                                    #9
    return pq
  else
    dist ← distance(node.point, target)                      #10
```